

Harden your Docker containers.

16 commands · production tested · CI ready

INSIDE

- SCANNING · 3**
scout cves · quickview · recommendations
- HARDENING · 5**
read-only · cap-drop · no-new-privs · seccomp
- USER & RES · 3**
non-root · pids · memory · cpus
- ISOLATION · 2**
internal networks · swarm secrets
- AUDIT · 3**
history · inspect · trust sign

swipe



Scout CVEs

01 / 16

Scan images for known vulnerabilities

```
zsh - docker scout cves

$ docker scout cves nginx:latest
✓ SBOM cached · 247 packages indexed
✗ 18 vulnerable · 34 CVEs total

Severity    1C    5H    12M    16L

CRITICAL    CVE-2022-37434    zlib 1.2.11
CRITICAL    CVE-2023-45853    zlib 1.2.11
HIGH        CVE-2023-0286     openssl 3.0.7
MEDIUM      CVE-2023-2650     openssl 3.0.7
LOW         CVE-2023-0464     openssl 3.0.7
```

FLAGS

- only-severity** → Filter by severity level
- only-fixed** → Show only CVEs with available fixes
- format sarif** → Export findings in SARIF format
- ignore-base** → Skip CVEs inherited from base image

USE CASE

Add to CI on every image build. Fail the pipeline on new critical or high findings to enforce a CVE ceiling.

PRO TIP

Use `--ignore-base` to focus only on team-owned deps.
Base image CVEs are upstream → patch by rebasing.

Quickview

02 / 16

One-screen image risk summary

```
zsh - docker scout quickview

$ docker scout quickview nginx:1.25
```

	Image	vulnerabilities				
Your image	nginx:1.25	0C	2H	8M	13L	
Base image	alpine:3.18	0C	1H	4M	3L	
Refresh base	alpine:3.19	0C	0H	2M	1L	(-1H -2M)
Update tag	nginx:1.25.4	0C	0H	5M	8L	(-2H -3M)

```

→ Policy compliant: PASS
→ Supply chain attestation: verified

```

FLAGS

- platform** → Target a specific OS/architecture pair
- exit-code** → Return non-zero exit on policy failure
- org** → Apply your organization's scout policies
- ref** → Compare against a baseline image ref

USE CASE

Run a fast pre-pull check on any image you don't own.
Cheaper than full CVE scans for daily compliance gates.

PRO TIP

Pipe `--exit-code` into your deploy pipeline. Any policy failure auto-blocks the release without manual review.

Scout Recs

03 / 16

Find the lowest-CVE upgrade path

zsh – docker scout recommendations

```
$ docker scout recommendations nginx:latest
```

Refresh base image

```
alpine:3.18 → alpine:3.20
```

removes 1 critical · 3 high

Update base image

```
alpine:3.18 → debian:12-slim
```

removes all critical · 5 high

Pin to digest

```
@sha256:e890a1...d2bb02 recommended
```

FLAGS

- tag** → Restrict to a single tag for analysis
- only-refresh** → Show only same-base refresh suggestions
- only-update** → Show only base image upgrade suggestions
- platform** → Limit to a specific OS/architecture

USE CASE

Pick the cheapest base image swap that removes the most CVEs without breaking glibc/musl or distro assumptions.

PRO TIP

Always pin by @sha256 digest, never floating tags. Tags are mutable. Digests are reproducible across builds.

Read - Only

04 / 16

Make the container filesystem immutable

```
zsh — docker run --read-only
```

```
$ docker run --read-only nginx:alpine
nginx: [emerg] mkdir("/var/cache/nginx")
      failed (30: Read-only file system)

$ docker run --read-only \
  --tmpfs /var/cache/nginx \
  --tmpfs /var/run \
  --tmpfs /tmp \
  nginx:alpine
2024/01/15 nginx ready on port 80
```

FLAGS

```
--read-only      → Mount the root filesystem as read-only
--tmpfs PATH     → Mount a writable tmpfs at the given path
-v VOL:PATH      → Bind a named volume for persistent writes
--mount type=    → Detailed mount spec (tmpfs / bind / volume)
```

USE CASE

Block attackers from dropping payloads, web shells, or persistence backdoors onto the container filesystem.

PRO TIP

Most apps only need tmpfs at /tmp, /var/run, and one or two cache dirs. Start there, then trim with strace logs.

Tmpfs Mounts

05 / 16

Writable scratch space, never on disk

```
zsh — docker run --tmpfs

$ docker run \
  --tmpfs /tmp:rw,size=128m,noexec,nosuid \
  --rm alpine

$ mount | grep tmpfs
tmpfs on /tmp type tmpfs
(rw,nosuid,noexec,size=131072k)

$ dd if=/dev/zero of=/tmp/big bs=1M count=200
dd: writing /tmp/big: No space left on device
```

FLAGS

size= → Hard cap on tmpfs allocation (e.g. size=64m)

noexec → Refuse to execute any file mounted on tmpfs

nosuid → Ignore setuid and setgid bits inside tmpfs

mode= → Set mount permissions (e.g. mode=1770)

USE CASE

Pair with `--read-only` to give apps a writable scratch dir without leaking state to disk between restarts.

PRO TIP

Always set size= and noexec. An unbounded tmpfs can exhaust host memory and OOM-kill unrelated containers.

Cap - Drop

06 / 16

Strip Linux kernel capabilities

```
zsh — docker run --cap-drop
```

```
$ docker run --cap-drop=ALL nginx:alpine
nginx: [emerg] bind() to 0.0.0.0:80 failed
(13: Permission denied)

$ docker run \
  --cap-drop=ALL \
  --cap-add=NET_BIND_SERVICE \
  nginx:alpine
nginx ready on port 80 (1 cap added, 13 dropped)
```

FLAGS

- cap-drop=ALL** → Drop every Linux capability by default
- cap-add=** → Re-add only the capabilities you need
- privileged** → Avoid · grants all caps and host access
- Common adds** → NET_BIND_SERVICE, CHOWN, SETUID, SETGID

USE CASE

Strip kernel-level powers so RCE inside the container can't load modules, change time, or hijack network ifs.

PRO TIP

Docker grants 14 caps by default. Most apps need 0-2.
Audit live caps with `capsh --print` inside the container.

No - New - Privs

07 / 16

Block setuid privilege escalation

```
zsh — docker run --security-opt

$ docker run \
  --security-opt=no-new-privileges:true \
  --user 1000:1000 \
  ubuntu sudo whoami

sudo: effective uid is not 0, is /usr/bin/sudo
on a file system with the nosuid option set

$ cat /proc/1/status | grep NoNewPrivs
NoNewPrivs:      1      (escalation blocked)
```

FLAGS

- no-new-privileges** → Block all setuid privilege gains
- user** → Pair with non-root UID for full effect
- cap-drop=ALL** → Layered defense: drop caps too
- read-only** → Pair with read-only rootfs for hardening

USE CASE

Stop attackers from regaining root via setuid binaries after you have set --user to a non-zero UID.

PRO TIP

Set this even on containers that already run as root.
Cheap insurance against setuid abuse in base images.

Seccomp

08 / 16

Filter the kernel sysctl surface

```
zsh - docker run --security-opt seccomp

$ docker run \
  --security-opt seccomp=strict.json \
  --rm alpine sh -c 'unshare --user'
unshare: unshare(0x10000000): Operation not permitted

$ docker info --format '{{.SecurityOptions}}'
[name=seccomp,profile=builtin]

# default profile blocks ~50 dangerous syscalls
# custom profile: 300+ syscalls denied
```

FLAGS

- seccomp=PATH** → Load a custom seccomp JSON profile
- seccomp=unconf** → Avoid · disables syscall filtering
- Default** → Built-in profile blocks ~50 syscalls
- apparmor=** → Layer with AppArmor for MAC controls

USE CASE

Shrink the kernel attack surface by blocking syscalls the app does not use (mount, kexec, ptrace, bpf, etc).

PRO TIP

Generate tight profiles with docker-slim or strace logs.
Manual JSON authoring is error-prone and breaks updates.

09 / 16

Drop root before running the app

```
zsh — docker run --user

$ docker run --user 1000:1000 alpine id
uid=1000 gid=1000 groups=1000

$ docker run --user 1000 alpine cat /etc/shadow
cat: can't open '/etc/shadow': Permission denied

# In Dockerfile (preferred):
RUN addgroup -S app && adduser -SG app app
USER app

# baked-in UID survives runtime --user overrides
```

FLAGS

- user UID:GID** → Run process as specific UID and GID pair
- user UID** → Avoid · GID defaults to 0 (root group)
- group-add** → Add supplementary groups at runtime
- Dockerfile USER** → Bake non-root into the image, not runtime

USE CASE

Stop container breakouts via shared volumes from being able to read or write host files as root by default.

PRO TIP

```
Bake USER 1000:1000 into the Dockerfile. Runtime --user
can break apps that chown directories on first start.
```

Pids Limit

10 / 16

Cap the number of processes

```
zsh - docker run --pids-limit

$ docker run --pids-limit=10 \
    alpine sh -c 'for i in $(seq 1 50);
                  do sleep 60 & done'

sh: can't fork: Resource temporarily unavailable

$ docker exec <id> cat \
    /sys/fs/cgroup/pids/pids.max
10    (enforced by cgroup v1/v2)

$ docker stats --format '{{.PIDs}}'
10 / 10
```

FLAGS

```
--pids-limit N      → Cap the maximum number of processes
--pids-limit -1     → Unlimited · Docker's risky default
cgroup pids.max     → Backing cgroup file enforcing the limit
Common range        → 50-500 covers most production workloads
```

USE CASE

Stop fork bombs and runaway worker leaks from eating all host PIDs and crashing other containers with them.

PRO TIP

Profile peak PID usage with `--pids-limit=10000` in dev.
Set production limit to 2x peak, never the raw peak.

Memory & CPU

11 / 16

Cap resource consumption

```
zsh - docker run --memory --cpus

$ docker run \
  --memory=256m --memory-swap=256m \
  --cpus=1.0 --cpu-shares=512 \
  nginx:alpine

$ docker stats --no-stream
CONTAINER    MEM USAGE / LIMIT    CPU %
abc123       45.2MiB / 256MiB     12.5%

# Container OOM-killed on exceed:
$ dmesg | grep oom · Killed nginx (256MB)
```

FLAGS

- memory M** → Hard ceiling on container memory usage
- memory-swap M** → Total memory + swap. Equal = no swap
- cpus N** → CPU count (fractional like 0.5 is valid)
- cpu-shares N** → Relative CPU weight, default 1024

USE CASE

Stop one noisy or compromised container from starving every other workload and crashing the entire host.

PRO TIP

Set --memory equal to --memory-swap to disable swap.
Faster OOM, no disk thrashing, predictable failure mode.

Internal Net

12 / 16

Networks with zero external reach

```
zsh - docker network create --internal

$ docker network create --internal backend
3f4a2b1c8e9d

$ docker run \
    --network backend --name db postgres:16

$ docker run --network backend \
    --rm alpine ping -c 1 google.com
ping: bad address 'google.com'

$ docker network ls
NAME          DRIVER          SCOPE          INTERNAL
backend       bridge          local          true
```

FLAGS

- internal** → Disable external connectivity entirely
- driver** → Bridge, overlay, macvlan, or ipvlan
- attachable** → Allow standalone containers to attach
- subnet** → Pin a specific CIDR for IP planning

USE CASE

Put your database and cache on a network that physically cannot reach the internet, even if the app is compromised.

PRO TIP

Use two networks: 'app' for internet-facing services and 'backend' --internal for databases reachable only from app.

Secrets

13 / 16

In-memory credentials, never on disk

```
zsh - docker secret create

$ echo 'supersecret' | \
  docker secret create db_pass -
abc123def456

$ docker service create \
  --name db --secret db_pass \
  postgres:16

$ docker exec <task> ls -l /run/secrets/
-r--r--r-- 1 root root 12 db_pass

# mounted as tmpfs · never written to disk
```

FLAGS

secret create NAME FILE	→ Create a new secret from a file
secret create NAME -	→ Create a new secret from stdin
--secret NAME	→ Mount the secret into a service
/run/secrets/NAME	→ Path where mounted inside container

USE CASE

Replace env-var passwords with tmpfs secrets that are isolated per-service and never leaked through docker ps.

PRO TIP

Secrets are Swarm-only. For Compose, declare secrets as external: true or migrate to HashiCorp Vault for parity.

History

14 / 16

Audit image layers for leaks

```
zsh — docker history --no-trunc

$ docker history --no-trunc \
  myapp:1.0 --format '{{.Size}}: {{.CreatedBy}}'

0B      CMD ["node" "server.js"]
0B      EXPOSE 3000
4.8MB   COPY package.json .
0B      ENV AWS_ACCESS_KEY=AKIA...8X9Q
12MB    COPY .env /app/.env      ← leaked!
72MB    ADD file:abc123 in /

# AWS key + .env still in image layers
```

FLAGS

- no-trunc** → Show complete layer commands, not truncated
- human** → Use human-readable byte sizes (default on)
- format** → Custom Go template output for parsing
- quiet** → Print only the layer IDs for scripting

USE CASE

Find leaked AWS keys, .env files, npm tokens, and SSH private keys that got accidentally COPY'd into images.

PRO TIP

Pipe to `grep -iE 'token|secret|password|key|aws_|bearer'`
for fast credential audits across every image you ship.

Inspect

15 / 16

Verify runtime security posture

```
zsh - docker inspect

$ docker inspect --format
    'User: {{.Config.User}}
    ReadOnly: {{.HostConfig.ReadonlyRootfs}}
    Caps: {{.HostConfig.CapAdd}}
    SecOpts: {{.HostConfig.SecurityOpt}}' \
api-prod
User:      1000:1000
ReadOnly:  true
Caps:      [NET_BIND_SERVICE]
SecOpts:   [no-new-privileges:true]
```

FLAGS

- `--format` → Go template syntax for shaping output
- `--format '{{json}}'` → Dump the full config as raw JSON
- `--type container` → Inspect runtime config of a container
- `--size` → Include disk size in container output

USE CASE

Verify that every container in prod actually matches your security baseline, not just what the manifest says.

PRO TIP

Save your security template once and diff every container against it. Drift detection in one shell pipeline.

Trust Sign

16 / 16

Cryptographically sign images

```
zsh - docker trust

$ export DOCKER_CONTENT_TRUST=1

$ docker trust key generate kerem
Generating key for kerem...
Private key: ~/.docker/trust/private/kerem.key

$ docker trust sign myorg/api:v1.0
Signing and pushing trust data...
v1.0: digest: sha256:abc123 signed

$ docker pull myorg/api:unsigned
Error: remote trust data does not exist
```

FLAGS

trust key generate	→ Create a new local signing key pair
trust signer add	→ Authorize a public key for a repo
trust sign IMG	→ Sign the image and push to registry
DOCKER_CONTENT_TRUST	→ Set =1 to enforce signed pulls only

USE CASE

Block unsigned images from ever reaching production by enforcing content trust on every docker pull and run.

PRO TIP

Notary v1 is deprecated. Migrate to cosign + sigstore for modern OCI signing with keyless transparency logs.

Save it. Ship secure.

RECAP

- Scan every image before it ships
- Drop all capabilities, add only what you need
- Never run as root · always set --user
- Limit pids · memory · cpus to prevent DoS
- Sign images · isolate networks · audit configs



@kerem.tech
Instagram



@keraattin
X



keraattin
GitHub



@keraattin
Kerem Tech